# 数据库综合实训 · 环境监测数据看板

## 佳芯科技 IMP-MC-01 离子注入工序数据平台

—— 项目概述与数据库原理 ——

（课程文档 · Word/PDF 交付版）

# 一、项目概述

## 1.1 项目是什么

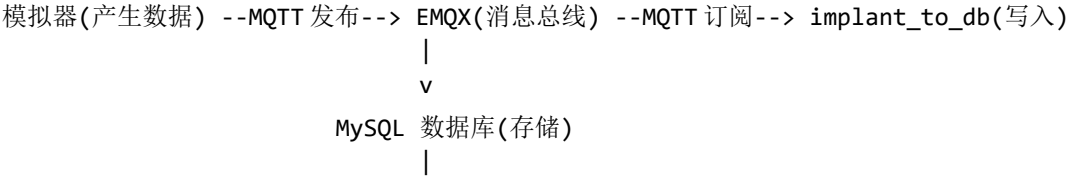
本项目是一套完整的环境监测数据平台，围绕一台「离子注入机（IMP-MC-01）」构建了从数据生产、传输、存储、服务到三维可视化的全链路系统。它模拟真实半导体生产环境：离子注入机持续加工晶圆，逐片产生工艺数据；这些数据经 MQTT 消息总线进入关系型数据库，再由后端接口提供给浏览器端，最终以「3D 数据看板」的形式直观呈现机台运行、晶圆传输、束流注入与报警检修的完整过程。

## 1.2 总体架构（数据链路）

**整个系统采用典型的分层架构，数据单向流动：**

| 环节 | 组件 | 职责 |
| --- | --- | --- |
| ① 数据源（生产） | Python 模拟器 implant_simulator.py | 模拟离子注入机逐片加工：生成晶圆工艺参数（能量/剂量/束流/均匀度等）、机台状态、报警事件 |
| ② 消息总线（传输） | EMQX（MQTT Broker，端口 1883） | 主题订阅/发布：模拟器作为生产者发布消息，数据库写入服务作为消费者订阅 |
| ③ 持久化（存储） | MySQL 8.0（库 fab_implant） | implant_to_db.py 订阅 MQTT 主题，将数据解析后写入 6 张业务表 |
| ④ 后端服务（访问） | Flask implant_app.py （端口 5001） | 提供 REST API（KPI/趋势/晶圆记录/报警/批次操作），从 MySQL 查询聚合后返回 JSON |
| ⑤ 反向代理（暴露） | Nginx（端口 80） | 将外部请求转发到 Flask 5001，静态资源（Three.js/GLB 模型/CSS）直接由 Nginx 分发 |
| ⑥ 前端（呈现） | 浏览器 3D 数据看板 | Three.js 加载 GLB 三维模型；操作电脑屏幕四页签实时刷新 KPI 与曲线 |

**数据流总览：**



v

Flask API(查询聚合) <--Nginx 反代<-- 浏览器 3D 看板

# 二、数据库在这个项目中发挥的作用

## 2.1 数据承载中枢（存储层）

数据库是整条链路的「中心枢纽」。模拟器产生的每一条数据（每片晶圆的工艺参数、每次机台状态切换、每次报警事件）都经 MQTT 汇总后落库。前端看到的任何数字（今日产量、良率、均匀度趋势、报警记录）都不是内存里的临时值，而是从数据库实时查询的结果——这保证了数据不丢、刷新一致、历史可追溯。

## 2.2 生产与展示解耦

由于数据全部落库，模拟器（生产端）与网页（展示端）互不依赖：模拟器可以暂停、重启、宕机，历史数据依然完整保留；网页刷新后看到的是历史全量数据而非丢失现场。这正是「数据库」相对「程序内变量/文件存储」的核心价值——生产端与消费端被数据库解耦。

## 2.3 支撑统计与聚合分析

KPI 面板的「今日产量」「7 日良率」「平均均匀度」，均匀度趋势折线图，批次报告的合格率/良率，全部依赖 SQL 的聚合查询（COUNT / AVG / GROUP BY / 时间窗口 JOIN）。数据库的 SQL 能力让统计无需在前端或模拟器里手动累加，而是交给关系引擎高效完成。

## 2.4 数据一致性保障

本项目在多处用到数据库的一致性能力：

- 唯一批次号（UNIQUE 索引）：一盒 20 片只写一条批次记录，重复关闭批次被幂等拦截；
- 报警开/关状态：end_time 为 NULL 表示未恢复报警，恢复时 UPDATE 而不是重复 INSERT；
- 机台状态时间线：每次状态切换 INSERT 一条，DOWN→RUNNING 转换时关闭所有未关闭报警；
- 凭据校验：登录从 operator_credentials 表验证工号密码，避免硬编码密钥散落在代码里。

# 三、数据库基本原理（结合本项目讲解）

## 3.1 什么是关系型数据库

关系型数据库（RDBMS）把数据组织成一张张「二维表」（Table），表由「行」（记录）和「列」（字段）构成，表与表之间通过公共字段（主键/外键）建立关系。本项目使用 MySQL 8.0，库名为 fab_implant，共 6 张业务表。

## 3.2 核心概念在本项目中的体现

| 概念 | 含义 | 本项目实例 |
| --- | --- | --- |
| 表（Table） | 同类数据的集合 | implant_wafer_records（逐片工艺记录）等 6 张表 |
| 行（记录） | 表中一条具体数据 | 一片晶圆的所有工艺参数 = 一行 |
| 列（字段） | 数据的某个属性 | energy_kev（能量）、dose_cm2（剂量）、uniformity_pct（均匀度） |
| 主键（PRIMARY KEY） | 唯一标识一行，非空 | 每张表的 id 自增主键；lot_id 唯一键 |
| 外键（FOREIGN KEY） | 引用另一张表的主键，建立关系 | wafer_records.lot_id 关联 lot_records.lot_id |
| 索引（INDEX） | 加速查询的目录结构 | created_at/state_time 等时间字段索引，加快按时间过滤 |
| 数据类型 | 规定列的存储格式 | DECIMAL(5,2) 存均匀度（精确小数）、DOUBLE 存剂量、DATETIME 存时间 |
| 事务（事务） | 一组操作要么全成要么全撤 | 批次写入/状态转换按「开始→结束」时间窗原子统计 |

## 3.3 六张表结构与 ER 关系

**表结构总览：**

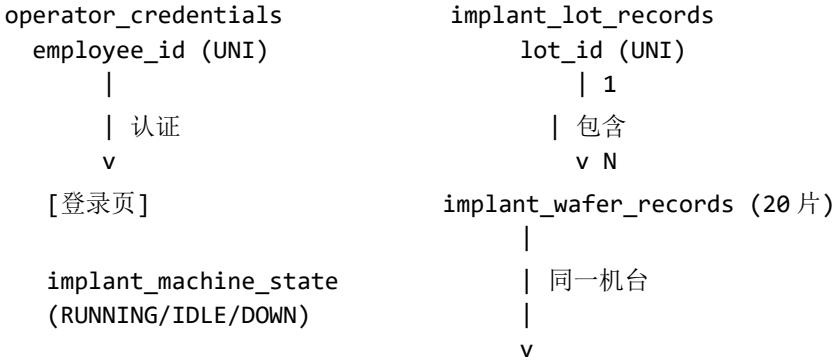
| 表名 | 行数 | 作用 | 关键字段 |
| --- | --- | --- | --- |
| implant_wafer_records | 17039 | 离子注入逐片工艺记录 | wafer_id, lot_id, species, energy_kev, dose_cm2, beam_current_ma, uniformity_pct, process_result, created_at |
| implant_machine_sta | 4388 | 机台状态时间线 | machine_id, |

| | | | |
|---|---|---|---|
| te | | | state(RUNNING/IDLE/DOWN/PM), reason, state_time |
| implant_alarm_events | 2383 | 报警事件（含恢复） | alarm_code, alarm_level(WARN/ERROR), message, start_time, end_time |
| implant_lot_records | 23 | 批次汇总（一盒一条） | lot_id(UNI), wafer_count, pass_count, yield_pct, uniformity_avg, alarm_count |
| implant_lot_sessions | 27 | 批次会话（开/关盒） | lot_id(UNI), total, start_ts, end_ts, state |
| operator_credentials | 3 | 操作员凭据 | employee_id(UNI), password, name, role, active |

**实体关系（ER）简述：**

- implant_lot_records（批次） 1 —— N implant_wafer_records（晶圆）：一个批次包含 20 片晶圆，通过 lot_id 关联；
- implant_lot_sessions（会话） 与 implant_lot_records（记录）共用 lot_id：会话记录「开始→结束」时间窗，记录在结束时按时间窗聚合统计；
- implant_machine_state（状态）与 implant_alarm_events（报警）均为机台层面的时间线数据：状态 DOWN 期间产生的 ERROR 报警，恢复时按时间窗关闭；
- operator_credentials（凭据）独立于业务数据，仅用于登录认证。

**ER 图（简）：**



```
                                implant_alarm_events
                                (WARN/ERROR + end_time)
    implant_lot_sessions: lot_id + start_ts/end_ts (会话时间窗)
```

## 3.4 MQTT → MySQL → API 的数据流（示例）

**以一片晶圆加工完成为例，数据经过完整链路：**

### 第一步：模拟器发布 MQTT 消息

```
# implant_simulator.py 中，每片晶圆加工完成后发布到主题
TOPIC_WAFER = 'fab/implant/wafer'
client.publish(TOPIC_WAFER, json.dumps({
    'lot_id': 'LOT-20261101-001', 'wafer_id': 'W-0001', 'slot_no': 1,
    'species': 'P', 'energy_kev': 50.0, 'dose_cm2': 1.0e15,
    'beam_current_ma': 8.5, 'uniformity_pct': 1.23,
    'process_result': 'PASS', 'created_at': '2026-11-01 10:23:45'
}))
```

### 第二步：implant_to_db.py 订阅并 INSERT

```
# implant_to_db.py 中
def on_wafer(client, userdata, msg):
    data = json.loads(msg.payload)
    cur.execute("""INSERT INTO implant_wafer_records
        (machine_id, lot_id, wafer_id, slot_no, species, energy_kev, dose_cm2,
         beam_current_ma, uniformity_pct, process_result, created_at)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)""",
        (MACHINE_ID, data['lot_id'], data['wafer_id'], data['slot_no'],
         data['species'], data['energy_kev'], data['dose_cm2'],
         data['beam_current_ma'], data['uniformity_pct'],
         data['process_result'], data['created_at']))
    c.commit()
```

### 第三步：Flask API 查询聚合

```
# implant_app.py /api/kpis 的核心查询
SELECT COUNT(*) FROM implant_wafer_records
WHERE DATE(created_at) = CURDATE()            -- 今日产量

SELECT AVG(uniformity_pct) FROM implant_wafer_records
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)  -- 7日平均均匀度

# /api/trend：取最新 80 条再反转（DESC 取最新 + reverse 保时间正序）
```

### 第四步：前端轮询渲染

浏览器 3D 页面每 5 秒调用 /api/kpis、/api/trend 等接口，把返回的 JSON 渲染到操作电脑屏幕的四页签（工艺汇总/晶圆质量/工序参数/操作）与 KPI 面板上。

## 四、技术栈与部署

| 层级 | 技术 | 版本/端口 | 说明 |
| --- | --- | --- | --- |
| **模拟器** | Python | 3.9.10 | 模拟离子注入工艺数据，--loop 模式由网页驱动节拍 |
| **消息总线** | EMQX | 4.4.19 / 1883 | MQTT Broker，控制台 18083，凭据 admin/Env2026!emqx |
| **数据库** | MySQL | 8.0.42 / 3306 | 库 fab_implant；root 密码 Env2026!root |
| **后端** | Flask + pymysql + paho-mqtt | 3.1.3 / 5001 | REST API + 登录会话 + 指令下发（MQTT publish_cmd） |
| **反向代理** | Nginx | 1.27.4 / 80 | 反代 /3d /2d /login 到 Flask，静态资源直发 |
| **前端** | Three.js r160 + GLB + 原生 JS | 浏览器 | 3D 场景、过场动画、操作电脑屏幕（screen_ui.js）、Glassmorphism 风格 |

**一键启动/停止：**

- 启动：双击 D:\dbtraining\启动离子注入网站.bat（依次拉起 MySQL → EMQX → Nginx → to_db → Flask → 模拟器）；
- 停止：双击 D:\dbtraining\停止离子注入网站.bat；
- 登录：工号 202301020772，密码 123456（数据库中 operator_credentials 表验证）。

—— 文档结束 ——